



Universidad Nacional Autónoma
de México



Facultad de Ciencias

Sistemas Operativos (7004)

Tarea 4: Bloqueos

Alumno: Hernández Flores Karina

Número de Cuenta: 317346404

Profesor: M.C. Javier León Cotonieto

Semestre: 2026 - 2

Fecha de entrega: 10 de marzo de 2026

1. Resumen: *Capítulo 8 - Deadlocks*

En los sistemas operativos modernos, múltiples procesos o hilos se ejecutan de manera concurrente y compiten por recursos limitados del sistema, como CPU, memoria, archivos o dispositivos de entrada y salida. Cuando varios procesos solicitan recursos que están ocupados por otros procesos, pueden entrar en estado de espera. En ciertos casos, esta situación puede provocar un **deadlock** o interbloqueo, en el cual un conjunto de procesos queda bloqueado indefinidamente porque cada uno espera que otro libere un recurso que necesita. Como resultado, ninguno de los procesos puede continuar su ejecución.

1.1. Modelo del sistema y uso de recursos

Un sistema informático dispone de distintos tipos de recursos, cada uno con un número determinado de instancias. Por ejemplo, pueden existir varios CPUs, archivos o dispositivos de red. Los procesos o hilos deben seguir una secuencia específica para utilizar estos recursos: primero deben **solicitarlos**, después **utilizarlos** y finalmente **liberarlos**. El sistema operativo mantiene información sobre qué recursos están disponibles y cuáles están asignados a cada proceso. Cuando un proceso solicita un recurso que está ocupado, se coloca en una cola de espera hasta que dicho recurso sea liberado.

Un deadlock ocurre cuando existe un conjunto de procesos donde cada uno está esperando un recurso que está en posesión de otro proceso del mismo conjunto. Un ejemplo clásico ocurre en aplicaciones multihilo donde se utilizan *mutex locks*. Si dos hilos adquieren recursos en distinto orden, puede suceder que cada uno posea un recurso y espere indefinidamente por el otro.

1.2. Livelock

Además del deadlock existe otro problema relacionado llamado **livelock**. En este caso, los procesos no están bloqueados, pero continúan ejecutando acciones repetitivas que no permiten avanzar en la ejecución del programa. Una analogía común es la de dos personas que intentan pasar por un pasillo moviéndose constantemente hacia un lado y otro sin lograr avanzar. Aunque no están detenidas, tampoco logran progresar.

1.3. Condiciones necesarias para el deadlock

Para que ocurra un deadlock deben cumplirse simultáneamente cuatro condiciones fundamentales:

- **Exclusión mutua**: al menos un recurso debe ser utilizado por un solo proceso a la vez.
- **Retención y espera (Hold and Wait)**: un proceso mantiene un recurso mientras espera obtener otros adicionales.
- **No apropiación (No Preemption)**: los recursos no pueden retirarse forzosamente; el proceso debe liberarlos voluntariamente.
- **Espera circular**: existe una cadena circular de procesos donde cada uno espera un recurso que posee otro proceso de la cadena.

Si cualquiera de estas condiciones no se cumple, el deadlock no puede ocurrir.

1.4. Grafos de asignación de recursos

Los deadlocks pueden representarse mediante un **grafo de asignación de recursos**, donde los procesos y los recursos se representan como nodos conectados por aristas. Si el grafo contiene un ciclo, puede existir un deadlock. Cuando cada recurso tiene una sola instancia, la presencia de un ciclo garantiza que el sistema está en estado de deadlock.

1.5. Métodos para manejar deadlocks

- **Ignorar el problema:** algunos sistemas operativos, como Linux y Windows, asumen que los deadlocks ocurren con poca frecuencia.
- **Prevención o evitación:** el sistema impone reglas para evitar que se cumplan las condiciones necesarias para el deadlock.
- **Detección y recuperación:** el sistema permite que el deadlock ocurra, pero implementa algoritmos para detectarlo y resolverlo.

1.6. Prevención del deadlock

La prevención consiste en garantizar que al menos una de las cuatro condiciones necesarias no pueda ocurrir. Un método común es imponer un **orden en la solicitud de recursos**, obligando a los procesos a solicitarlos en un orden específico. De esta manera se evita la formación de una espera circular.

Sin embargo, estos métodos pueden disminuir la eficiencia del sistema, ya que los procesos podrían mantener recursos que no utilizan durante largos periodos o esperar innecesariamente.

1.7. Evitación del deadlock

La evitación requiere que el sistema tenga información previa sobre las necesidades máximas de recursos de cada proceso. Con esta información, el sistema puede decidir si conceder o no una solicitud sin poner al sistema en un estado inseguro.

El algoritmo más conocido para este propósito es el **algoritmo del banquero**. Este algoritmo determina si el sistema permanece en un *estado seguro* después de conceder recursos. Un estado seguro es aquel en el que existe al menos una secuencia de ejecución en la que todos los procesos pueden completar sus tareas sin provocar un deadlock.

1.8. Detección y recuperación

Cuando no se aplican técnicas de prevención o evitación, el sistema puede utilizar algoritmos para **detectar deadlocks**. Estos algoritmos analizan las relaciones de espera entre procesos mediante estructuras como el *wait-for graph*, buscando ciclos que indiquen interbloqueo.

Una vez detectado el deadlock, el sistema puede recuperarse mediante dos estrategias principales:

- **Terminar uno o más procesos** involucrados en el deadlock.
- **Preemptar recursos**, es decir, retirarlos temporalmente de algunos procesos para romper el ciclo de espera.

```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
    pthread_mutex_lock(&first_mutex);
    pthread_mutex_lock(&second_mutex);
    /**
     * Do some work
     */
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);

    pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
    pthread_mutex_lock(&second_mutex);
    pthread_mutex_lock(&first_mutex);
    /**
     * Do some work
     */
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);

    pthread_exit(0);
}
```

Figura 1: Deadlock example.

2. Ejercicio

8.13 Draw the resource-allocation graph that illustrates deadlock from the program example shown in Figure 8. 1 in Section 8.2.

En este ejemplo participan dos hilos:

- $T_1 = \text{thread_one}$
- $T_2 = \text{thread_two}$

y dos recursos tipo mutex:

- $R_1 = \text{first_mutex}$
- $R_2 = \text{second_mutex}$

El comportamiento de los hilos es el siguiente:

- `thread_one` adquiere primero `first_mutex` y después intenta adquirir `second_mutex`.
- `thread_two` adquiere primero `second_mutex` y después intenta adquirir `first_mutex`.

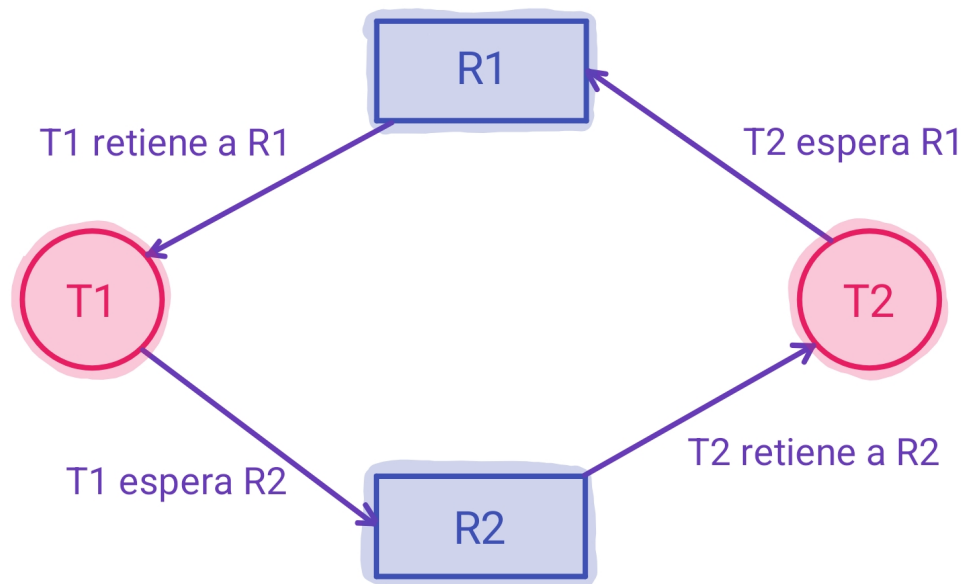


Figura 2: Gráfico de Asignación de Recursos de la Figura 1

El *deadlock* ocurre cuando:

- $R_1 \rightarrow T_1$: `first_mutex` está asignado a `thread_one`.
- $T_1 \rightarrow R_2$: `thread_one` está esperando `second_mutex`.
- $R_2 \rightarrow T_2$: `second_mutex` está asignado a `thread_two`.
- $T_2 \rightarrow R_1$: `thread_two` está esperando `first_mutex`.

Por lo tanto, el grafo forma el ciclo:

$$R_1 \rightarrow T_1 \rightarrow R_2 \rightarrow T_2 \rightarrow R_1$$

Este ciclo demuestra la existencia de una **espera circular**. Como cada recurso tiene una sola instancia, la presencia del ciclo implica que el sistema se encuentra en estado de *deadlock*.

Gráfico de Asignación de Recursos

En la Figura 2, se observa el gráfico de asignación de recursos que muestra que ambos hilos quedan bloqueados indefinidamente, ya que cada uno espera un recurso retenido por el otro.

Punto extra: simulación en código

Se implementó una simulación en lenguaje C del grafo de asignación de recursos. En el programa se representan dos hilos (T_1 y T_2) y dos recursos (R_1 y R_2), junto con las relaciones de asignación y solicitud descritas en el ejemplo del capítulo.

La simulación construye el grafo mediante una matriz e imprime las aristas correspondientes. además, realiza una búsqueda para detectar si existe un ciclo. Dado que el programa identifica un ciclo en el grafo, concluye que el sistema se encuentra en estado de *deadlock*.

Esta implementación permite comprobar mediante código la misma condición de deadlock que se representa gráficamente en la solución teórica del ejercicio.

A continuación, se encuentra el código de esta implementación.

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define NODES 4
5
6  // Indices de nodos
7  // 0 = T1, 1 = T2, 2 = R1, 3 = R2
8
9  const char *names[NODES] = {"T1", "T2", "R1", "R2"};
10
11 int graph[NODES][NODES] = {0};
12
13 void add_edge(int from, int to) {
14     graph[from][to] = 1;
15 }
16
17 void print_graph() {
18     printf("Grafo de asignacion de recursos:\n");
19     for (int i = 0; i < NODES; i++) {
20         for (int j = 0; j < NODES; j++) {
21             if (graph[i][j]) {
22                 printf("  %s -> %s\n", names[i], names[j]);
23             }
24         }
25     }
26 }
27
28 int visited[NODES];
29 int recStack[NODES];
30
31 int has_cycle_util(int node) {
32     visited[node] = 1;
33     recStack[node] = 1;
34
35     for (int neighbor = 0; neighbor < NODES; neighbor++) {
36         if (graph[node][neighbor]) {
37             if (!visited[neighbor]) {
38                 if (has_cycle_util(neighbor)) {
39                     return 1;
40                 }
41             } else if (recStack[neighbor]) {
42                 return 1;
43             }
44         }
45     }
46 }
```

```
44     }
45 }
46
47 recStack[node] = 0;
48 return 0;
49 }
50
51 int has_cycle() {
52     memset(visited, 0, sizeof(visited));
53     memset(recStack, 0, sizeof(recStack));
54
55     for (int i = 0; i < NODES; i++) {
56         if (!visited[i]) {
57             if (has_cycle_util(i)) {
58                 return 1;
59             }
60         }
61     }
62     return 0;
63 }
64
65 int main() {
66     // Asignaciones:
67     // R1 -> T1
68     // R2 -> T2
69     add_edge(2, 0);
70     add_edge(3, 1);
71
72     // Solicitudes:
73     // T1 -> R2
74     // T2 -> R1
75     add_edge(0, 3);
76     add_edge(1, 2);
77
78     print_graph();
79
80     if (has_cycle()) {
81         printf("\nSe detecto un ciclo en el grafo.\n\n");
82         printf("Conclusion: existe DEADLOCK.\n");
83     } else {
84         printf("\nNo se detecto ningun ciclo.\n");
85         printf("Conclusion: no hay deadlock.\n");
86     }
87
88     return 0;
89 }
```

```
Grafo de asignacion de recursos:  
T1 -> R2  
T2 -> R1  
R1 -> T1  
R2 -> T2  
  
Se detecto un ciclo en el grafo.  
Conclusion: existe DEADLOCK.
```

Figura 3: Resultado de la Simulación

En la Figura 3 se puede observar el resultado de la simulación.

A continuación, se adjunta el link al repositorio de GitHub:

<https://github.com/KarinaHF3/SistemasOperativos/tree/4185971de27538b974eef47eed2d7b5affee7f38/Tarea4>